

Problem 1.

- Analysis

To reach the goal of this problem, which is to identify all “peaks of interest” in a video. The algorithm must find the local maximum indices of an array containing intensity values, where each index corresponds to one second. Each element of the array is an integer between 0 and 100, a higher number represents a more interesting moment in the video.

In real-world video data, short fluctuations or small noise spikes may flag a false peak if only immediate comparisons are used. Therefore, to accurately specify what a peak in the array is, I assume that it is defined as a point where the intensity exhibits a clear rising trend before the point and a clear falling trend after the point.

To deal with the plateau region, I treat the first index of the plateau as the representative peak, provided that the general rising and falling patterns still occur outside the flat region.

The output will be a list of indices that satisfy the rising-plateau-falling pattern and surpass the intensity threshold. Because the algorithm scans the array once and evaluates a fixed-size neighborhood around each element, it is possible to achieve a linear-time solution.

- Design

The algorithm scans the intensity array and checks each index as a potential peak. It begins with a simple threshold check: if the value at the current index is 10 or below, the index cannot represent meaningful interest and is skipped immediately.

Next, it determines whether the index begins a plateau by scanning forward until the intensity changes; the first index of this flat region is treated as the candidate peak. To verify that the signal rises to this point, the algorithm checks that the intensity at $i-1$ is strictly smaller than the value of the current index. The values two and three seconds earlier must form a non-strictly increasing sequence.

For the falling trend, similarly, the value of the current index must be strictly greater than the first value after the plateau, while the next two and three seconds must form a non-strictly decreasing sequence. These conditions ensure that the peak represents a genuine transition from rising to

falling intensity. If all conditions are satisfied, the index is added to the list of peaks.

The algorithm completes a single pass over the array. Since each plateau is processed once and each check uses constant time, the overall runtime is $O(n)$.

- Justification

In the algorithm, the strict comparisons on the immediate neighbors ensure that any identified peak represents a feature of first rising, then falling, so the result aligns with the intuitive meaning of a peak. The use of non-strict comparisons within the wider window allows the algorithm to accommodate the natural variability of real-world video.

The plateau handling ensures completeness by capturing the same high value while avoiding false positives within the plateau itself. The algorithm checks all valid indices, so it does not miss any possible peaks under the defined conditions. Because it evaluates only a fixed number of surrounding values per index, its linear-time complexity is optimal and appropriate for processing video-length arrays.

Problem 2.

- Analysis

The goal of the algorithm is to merge two priority queues into a single priority queue. According to the K document, a priority queue is implemented as a heap-ordered array stored in a complete binary tree shape. In order to maintain heap order, swim and sink are used to implement insertion and deletion, and these operations run in $O(\log n)$ time. Keys are assumed to be unique, and the maximum is always at the first index.

Assuming two priority queues are A1 with size $N1$ and A2 with size $N2$. The result of the algorithm should be a priority queue R containing all elements from A1 and A2.

- Design

First, creating a new empty priority queue R has size $N_1 + N_2$. It copies all elements of A_1 into the first N_1 positions of R . Then it copies all elements of A_2 into the next N_2 positions.

Now, R is a Complete Binary Tree but not a heap. To rebuild the heap, the algorithm performs a bottom-up $\text{sink}()$ operation from K to rebuild it. It iterates from index $(N_1 + N_2)/2$ to 1 and calls $\text{sink}(k)$ on each index. Since all leaf nodes (indices $> (N_1 + N_2)/2$) do not have a child, and each internal node from the upper layer will sink at most once, the nodes from the next upper layer will sink at most twice, etc. Eventually, makes the entire reconstruction process run in $O(N_1 + N_2)$ time.

Once all sink operations have been applied, R becomes a valid max-heap priority queue that satisfies all the properties in K : it is stored as a complete binary tree, its root contains the maximum key, and every parent key is greater than or equal to its children.

- Justification

Firstly, the correctness of the algorithm follows from two facts about heap-ordered arrays. Copying the elements from A_1 and A_2 into a single array guarantees that result R contains all the elements. Also, applying $\text{sink}()$ bottom-up ensures that the result R is a valid heap construct.

Secondly, completeness is guaranteed because every element in the result R is considered during $\text{sink}()$. The algorithm is also consistent with the K document: it uses the same heap construct and the same operations, such as $\text{sink}()$.

Finally, the algorithm is efficient. The bottom-up heap construction takes $O(N_1 + N_2)$, which is optimal because it must at least inspect every element once. Therefore, this design provides a fast merge under the heap model specified by the K document.